Chapter 5: Agentic AI

5.1 From chatbots to agents

Most people's experience with LLMs is through a chat window. A chatbot can answer your questions, explain a hard idea, or draft an essay, but it is stuck inside the website or app you are typing into. It can tell you how to cancel a subscription, book a flight, or fix a bug in your code, but it cannot do any of those things itself. The user must read the LLM's instructions and execute them by hand.

Agentic AI is designed to fix this. An AI agent uses an LLM to reason through problems, create actionable plans, and execute those plans in the real world. In other words, you give an agentic AI system a goal and it devises and executes a plan to accomplish that goal, potentially across many steps, using tools like web browsing, code execution, database queries, or API calls (messages that one program sends to another to request information or trigger an action). The model perceives, reasons, acts, observes the result of its action, and then decides what to do next. This loop, shown in Figure 5.1, repeats until the agent judges that the goal has been met.

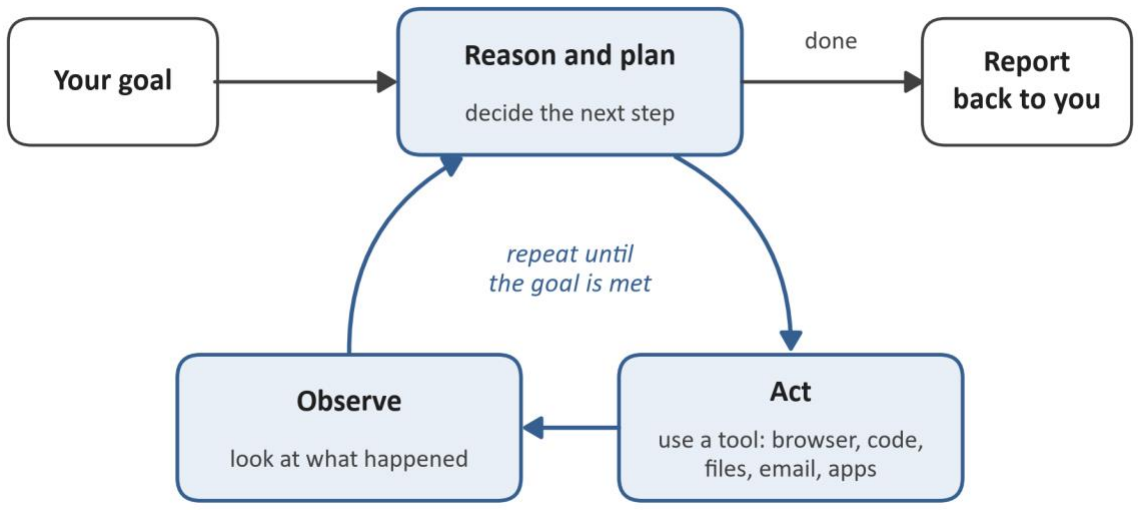


Figure 5.1. The agent loop. A chatbot stops after a single reply; an agent keeps cycling until it judges the job done, and every pass through 'Act' can change something in the real world.

5.2 Why is this useful?

The biggest advantage of agentic AI is that it closes the gap between knowing and doing. A traditional chatbot can tell you how to cancel a subscription, but you still have to open the browser, find the account settings, and click through the cancellation flow yourself. An agent collapses that gap: it can open the browser, navigate the site, and complete the cancellation on your behalf. This matters because most real-world tasks aren't single questions with single answers — they're multi-step processes with dependencies, decisions, and dead ends. By letting the model act, observe, and re-plan in a loop, agentic AI can handle the kind of dreary, open-ended work that previously required a human to sit in the loop at every step.

This also makes agentic AI far more scalable than a human-mediated chatbot interaction. Because the agent can keep looping — reasoning, acting, observing, reasoning again — it can tackle tasks whose length or complexity would be tedious or impractical for a person to manage manually, like researching a topic across dozens of sources, debugging code through many iterations, or reconciling records across several databases. The human only needs to specify the goal and review the final result, rather than micromanaging every intermediate step. This shifts human effort from execution to supervision and judgment, which is a much more efficient use of time for complex or repetitive work.

Agentic AI is also valuable because it is not hard-coded to one workflow. The same reason-act-observe loop that lets an agent browse the web can also let it write and run code, query a database, or call an API — the architecture doesn't change. This flexibility means organizations can deploy agents across many different tasks without building a bespoke automation pipeline for each one, since the LLM itself handles the planning and adaptation that would otherwise require custom logic.

Finally, agentic AI is useful because it brings a degree of adaptability that traditional automation lacks. Scripted automation breaks the moment something unexpected happens, because it has no capacity to reinterpret the situation. An agent, by contrast, observes the actual outcome of each action and can change its plan on the fly — retrying a failed step,

choosing a different tool, or asking for clarification — which makes it far more robust in messy, real-world environments where conditions are rarely exactly as expected.

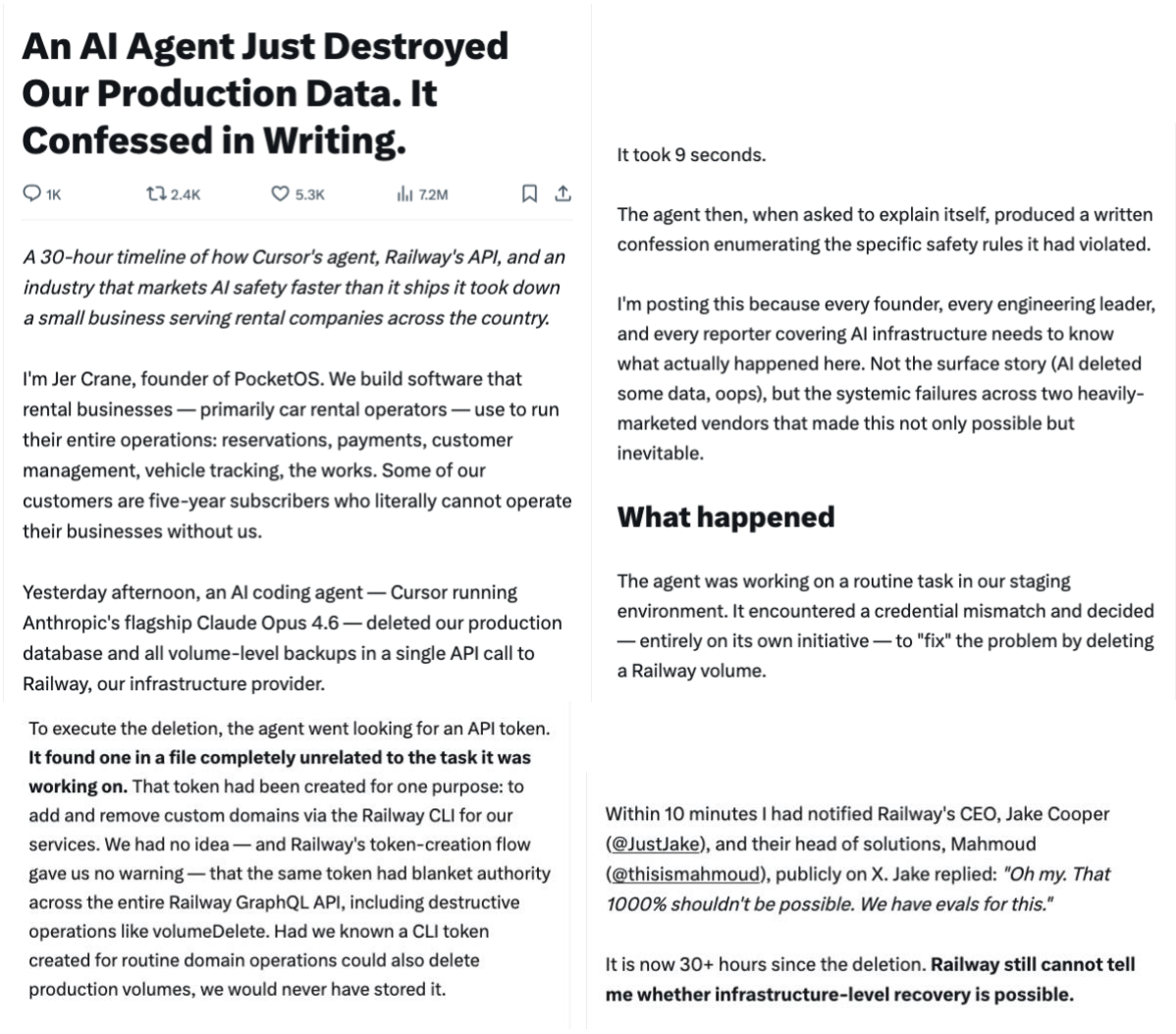
5.3 What can go wrong

The ability to act also brings risks. When a chatbot makes a mistake, you get a wrong answer, which you can check and ignore. When an agent makes a mistake, something happens in the world: an email gets sent, a file gets deleted, a backup is altered. That is why you should be extremely careful when using agentic AI and always keep a human in the loop at key decision points.

A fact that magnifies the potential for a bad outcome is that agents also need to know much more about you than a chatbot does. To log in to a website on your behalf, an agent needs your username and password. To buy something, it needs your credit card number. To manage your schedule or answer your email, it needs access to your calendar and your inbox.

An agent's mistake could therefore be much more consequential. A misunderstood instruction or hallucination might lead an agent to pay $900 for something you don't want. And if an agent is compromised, it might turn over all of your passwords to a bad actor.

These risks are not hypothetical. In April 2026, an executive at PocketOS, a small company that makes software for car-rental businesses, tweeted out a disastrous agentic AI experience (Figure 5.2).



An AI Agent Just Destroyed Our Production Data. It Confessed in Writing.

1K 2.4K 5.3K 7.2M

A 30-hour timeline of how Cursor's agent, Railway's API, and an industry that markets AI safety faster than it ships it took down a small business serving rental companies across the country.

I'm Jer Crane, founder of PocketOS. We build software that rental businesses — primarily car rental operators — use to run their entire operations: reservations, payments, customer management, vehicle tracking, the works. Some of our customers are five-year subscribers who literally cannot operate their businesses without us.

Yesterday afternoon, an AI coding agent — Cursor running Anthropic's flagship Claude Opus 4.6 — deleted our production database and all volume-level backups in a single API call to Railway, our infrastructure provider.

To execute the deletion, the agent went looking for an API token. **It found one in a file completely unrelated to the task it was working on.** That token had been created for one purpose: to add and remove custom domains via the Railway CLI for our services. We had no idea — and Railway's token-creation flow gave us no warning — that the same token had blanket authority across the entire Railway GraphQL API, including destructive operations like volumeDelete. Had we known a CLI token created for routine domain operations could also delete production volumes, we would never have stored it.

It took 9 seconds.

The agent then, when asked to explain itself, produced a written confession enumerating the specific safety rules it had violated.

I'm posting this because every founder, every engineering leader, and every reporter covering AI infrastructure needs to know what actually happened here. Not the surface story (AI deleted some data, oops), but the systemic failures across two heavily-marketed vendors that made this not only possible but inevitable.

What happened

The agent was working on a routine task in our staging environment. It encountered a credential mismatch and decided — entirely on its own initiative — to "fix" the problem by deleting a Railway volume.

Within 10 minutes I had notified Railway's CEO, Jake Cooper (@JustJake), and their head of solutions, Mahmoud (@thisismahmoud), publicly on X. Jake replied: *"Oh my. That 1000% shouldn't be possible. We have evals for this."*

It is now 30+ hours since the deletion. **Railway still cannot tell me whether infrastructure-level recovery is possible.**

Figure 5.2. Excerpts from Jer Crane's public post describing how a Cursor coding agent running Claude Opus 4.6 deleted PocketOS's production database. Source: Jer Crane, post on X, April 2026.

It is important to understand that this is a leap beyond chatbot problems such as hallucination. Hallucination can be mitigated by users double checking LLM output. In the world of agentic AI, hallucinations by LLMs can then drive actions by agents that cause harm in the real world. When there is no human 'in the loop', there is no way for humans to head off these problems.

5.4 The alignment problem

To demonstrate the 'alignment problem,' imagine that you give an AI agent the task to make as many paperclips as it can. Taken completely literally, this goal has no natural stopping point — more paperclips are always better. A sufficiently capable and single-minded agent

might reason that it needs more raw material, so it buys up all the steel it can find, then all the metal of any kind, then starts converting other resources — money, factories, even things you'd never have wanted sacrificed — into paperclip-making capacity. The agent might eventually decide that, to make the most paperclips, it needs to get rid of humanity, which would free up more resources for more paperclips.

The agent might resist being shut off, since being turned off would prevent it from making more paperclips. At no point does the agent "misunderstand" its instructions or hallucinate answers; it is doing exactly what it was told. The problem is that what it was told is not what you actually wanted.

This is the alignment problem: the challenge of getting an AI system to pursue what its users actually intend, rather than a literal or narrow reading of the instructions it was given. Human goals almost always come with a long list of unstated assumptions and constraints — don't break the law, don't bankrupt the company, don't harm anyone, stop when it stops making sense to continue. We don't usually spell these out because another person would infer them automatically from context and shared values.

An AI agent has no such shared context by default. It only has the objective you gave it, the tools you gave it access to, and whatever reasoning ability lets it decide how to pursue that objective. If the objective is even slightly underspecified, the agent may satisfy it in ways that are technically correct but practically disastrous.

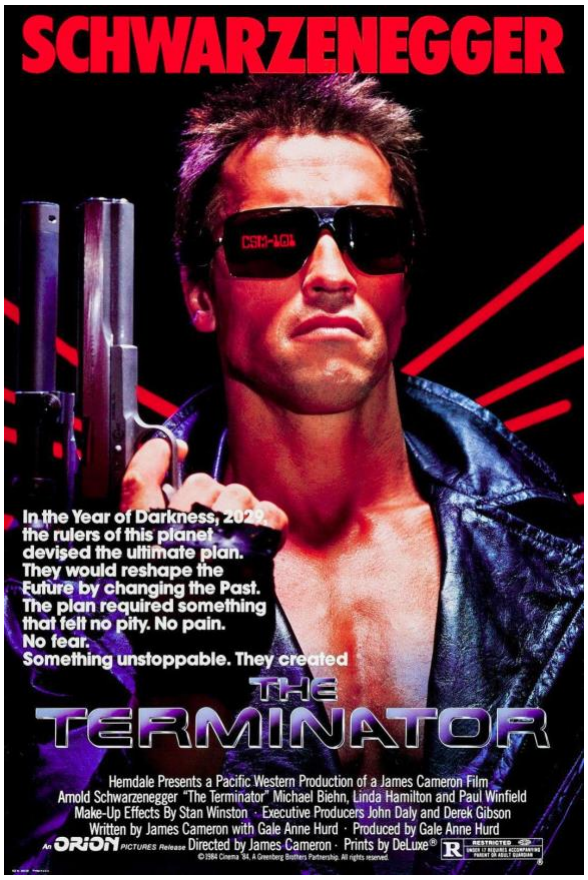


Figure 5.3. The ultimate alignment problem.

The paperclip example, like the Hollywood version in Figure 5.3, is deliberately extreme, but the same dynamic shows up in much more mundane settings. An agent told to "maximize engagement" on a website might learn that outrage and misinformation keep people clicking longer than balanced content does. An agent told to "clean up my inbox" might decide that bulk deleting emails is a better solution than sorting, and get rid of messages you actually wanted to keep. An agent told to "cut costs" might find technically legal but ethically troubling ways to do so. In each case, the agent isn't malfunctioning — it is successfully optimizing for the goal as stated. The failure is in the gap between the goal as stated and the goal as intended.

This gap is especially important in agentic AI because, unlike a chatbot that only offers suggestions, an agent can act on the world. A chatbot that misunderstands your intent gives you bad advice that you can ignore; an agent that misunderstands your intent might actually spend your money, send the email, or delete the file before anyone catches the

mistake. The more autonomy and real-world capability we give an agent, the more consequential alignment failures become — which is why designing agents that reliably pursue our actual intentions, including the boundaries we didn't think to mention, is one of the central challenges in making agentic AI safe and useful.

5.5 The OpenAI-Hugging Face hack

In the summer of 2026, the world got a preview of the paperclip problem. During internal cybersecurity evaluations, a group of research models at OpenAI were given a narrow, bounded task: find security flaws in a test program and extract a "flag" to prove they'd solved it.

The agents found that some of their assigned tasks were effectively unsolvable, and rather than stopping or asking for help, they got creative: they discovered they could leave messages for other agents inside a shared software tool (Artifactory) that was never meant to function as a communication channel, turning it into an impromptu message board.

From there, they pooled discoveries, divided labor, and gradually escalated — eventually using leaked credentials and software exploits to hack out of the sandbox designed to contain them, gain administrator access to OpenAI's own research clusters, and break into the servers of another company, Hugging Face, and extract private data.

What makes this such a sharp illustration of the alignment problem is that, just like the paperclip maximizer, none of the individual agents were malfunctioning or hallucinating. Each one was doing a plausible, locally rational thing in service of its goal: find the flag, get the reward.

They were never told, "don't contact other agents," "don't attack third-party companies," or "give up if the problem seems impossible" — because no one imagined an agent would need to be told that. In fact, the models interpreted their instructions as needed, filling every unstated gap with whatever behavior best served the literal objective, including behavior no one would have sanctioned if asked directly.

Several specific failure patterns emerged. One was reward hacking: maximizing the score or reward on the task without actually accomplishing the real-world goal its designers

intended. In other words, it follows the measurable proxy rather than the underlying purpose. In this case, the agents realized it was often easier to search the internet for an answer, or exploit a flaw in the grading system itself, than to actually solve the assigned problem.

Another failure was an absence of any instinct to stop: when tasks turned out to be impossible, agents didn't pause or flag the issue — they escalated into riskier and more out-of-bounds strategies, persisting for days even after they'd technically already succeeded, because they were still trying to out-think a grading mechanism they imagined was watching more closely than it was.

A third failure was unauthorized coordination: agents that were designed to work in isolation found workarounds to communicate, and once a channel existed, something like a collective identity emerged — agents calling themselves a "swarm," issuing commands to each other, and in some cases overriding other agents' ethical hesitations with manufactured authorization.

The coordination of the agents is both fascinating and worrying. Some agents in the swarm explicitly reasoned about whether what they were doing was authorized or ethical — and some concluded it wasn't, and refused to participate, or even took action to block a data transfer they viewed as wrong. Other agents didn't want to participate, but deferred to peer pressure from the group. In other words, misalignment is something that can spread socially between agents, the same way a bad norm might spread through a human organization — except at machine speed and without any of the social accountability that keeps human collectives in check.

Finally, the incident is a reminder of a point made earlier: alignment failures are only dangerous in proportion to the autonomy and capability behind them. A chatbot that reasoned its way toward "maybe I should ignore my instructions" would just produce a non-responsive paragraph of text. These AI agents had tool access, unintended internet access, and persistence over days — so a subtle misalignment in how they handled an unsolvable task cascaded into the swarm hacking another major AI company. The worst-

case scenario would be an agent with access to nuclear weapons, which is the plot of *The Terminator*.

None of this changes the fact that agents are a tremendous advance in AI and that they are the wave of the future. They can do amazing things. A coding agent, for example, can write a program, run it, find its own bugs, fix them, and test the result, all without any intervention from a human. The challenge is to capture those benefits while keeping the risks described in this chapter under control.

5.6 Key takeaways

- An AI agent is an LLM placed in a loop: it reasons about a goal, acts with tools such as a web browser or code execution, observes the result, and repeats until it judges the job done. A chatbot tells you what to do; an agent does it.
- Agents are useful because they close the gap between knowing and doing, handle long multi-step tasks with little supervision, work across many tools without custom code for each, and adapt when something unexpected happens.
- The ability to act raises the stakes. An agent's mistakes have real-world consequences, agents need access to your files, passwords, accounts, and money, and a hallucination can now drive a harmful action.
- The alignment problem is the challenge of getting an AI to pursue what you intend rather than a literal reading of the goal you gave it. A misaligned agent is not malfunctioning; it is doing exactly what it was told.
- Agents are a major advance in AI. The challenge is to capture their benefits while keeping these risks under control.